

Paradigmes de Programmation
M1/GL/S1/2025/2026
Série N° 04 (Paradigme fonctionnel) (Durée: 3H.)

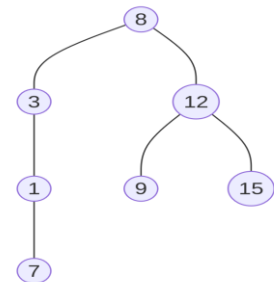
Exercice 1.

1. Définis un type jour représentant les 7 jours de la semaine;
2. Écrire les fonctions suivantes :
 - a) jour_suivant: Renvoie le jour suivant: (Dimanche → Lundi, Lundi → Mardi ..., Samedi → Dimanche);
 - b) est_weekend: Renvoie true si c'est Vendredi ou Samedi, sinon false;
 - c) jour_de_travail: Renvoie true si le jour est un jour de travail (Dimanche à jeudi) sinon false.
 - d) jours_travail: Qui prend une liste de jours et renvoie uniquement les jours de travail.
 - e) avance_jours : Elle prend un jour et un nombre n, et renvoie le jour obtenu après n jours. **Exemple** :
avance_jours Vendredi 3 = Lundi. avance_jours Vendredi 10 = Lundi.
 - f) jours_entre: Renvoie la liste de tous les jours entre deux jours inclus. **Exemple**: jours_entre Vendredi Mardi = [Vendredi; Samedi; Dimanche; Lundi; Mardi].
 - g) compte_jours: Prend une liste de jours et renvoie le nombre d'occurrences de chaque jour. **Exemple**:
compte_jours [Lundi; Lundi; Mardi; Lundi] = [(Lundi,3); (Mardi,1)]

Exercice 2

Un arbre binaire est une structure de données composée de nœuds :

- Chaque nœud contient une valeur et peut avoir jusqu'à deux enfants : un gauche et un droit.
- Un arbre peut être vide



1. Proposer un type pour cette structure de données;
2. Écrire la fonction taille qui renvoie le nombre total de nœuds dans l'arbre;
3. Écrire la fonction nbre_feuilles qui calcule le nombre de feuilles dans un arbre binaire;
4. Écrire la fonction hauteur qui renvoie la hauteur d'un arbre (hauteur d'un arbre vide est 0);
5. Écrire trois fonctions pour parcourir l'arbre et renvoyer une liste des valeurs :
 - a) parcours_prefixe (pré-ordre): Racine → Gauche → Droite;
 - b) parcours_infixe (in-ordre): Gauche → Racine → Droite;
 - c) parcours_suffixe (post-ordre): Gauche → Droite → Racine.